MAT1510: Final Project Report

Richard Shi
rshi@cs.toronto.edu
University of Toronto

1 Summary: Reinforcement Learning Finetunes Small Subnetworks in Large Language Models

Reinforcement Learning is a crucial post-training step for large language models (LLMs) by allowing them to complete complex reasoning tasks, and align with human values and safety protocols. These behaviours differ drastically from the original pretrained model so it is assumed that significant parameter changes and thus finetuning of the whole model is required [1]. This paper by Mukherjee et al. aims to shed light on whether finetuning actually updates the whole model and presents evidence to suggest that it does not.

Main result and conjecture. Suprisingly, the parameter updates from RL finetuning only updates a small subset of the whole model (the authors call this a "subnetwork"). The authors observed this in all RL algorithms studied which include PPO, GRPO, ORPO, KTO, DPO, SimPO, and PRIME [1]. Based on their findings, they also conjecture that performing finetuning only on this subnetwork while freezing the remaining parameters can produce a near-identical model to the full-finetuned model.

Update sparsity. The paper analyzes the conventional three-stage pipeline which consists of pre-training, supervised finetuning (SFT), and then RL finetuning. They find that the RL updates are sparse but full-rank while SFT produces dense updates. In addition, they find that these updates are distributed throughout transformer layers (rather than localized to specific weights) and that sparsity levels are similar throughout transformer layers.

Subnetwork performance. The authors compare the performance of full-finetuned model on the test set to a model trained with the same configuration (data, hyperparameters, and gradient steps) except that only the subnetwork can receive parameter updates. They find that the subnetwork-finetuned model achieves the same or better performance on the test set.

Consistency of subnetworks across different training configurations. The authors set out to discover whether the identified subnetworks persist across different training configurations. Surprisingly, there is a substantial overlap (more than their defined random baseline) in the subnetworks that emerge from different training seeds, data, and RL algorithm.

Emergence of subnetworks. Finally, the authors aim to identify factors that contribute to the update sparsity in RL. Gradient clipping and KL regularization prohibit large parameter updates so they should lead to update sparsity but experiments with and without these regulariation terms lead to similar update sparsity. Performing SFT on the same data before RL may lead to sparsity but models like `DeepSeek-R1-Zero` skip the SFT stage and go straight into RL finetuning. Training duration was also investigated and while update sparsity decreases initially, it shows signs of convergence. The authors posit that the models may be undertrained in their analysis but this hypothesis lacks foundation as it contradicts current RL literature which argues against overtraining to prevent overfitting and improve generalization [1]. Only training on in-distribution data (data drawn from a distribution closely matching the current policy) was found to be the primary factor of this update sparsity. This is done by either using an on-policy RL scheme or performing SFT on the same data used for subsequent RL.

Conclusion. This work highlights potential avenues for optimizing RL training for LLMs through the finetuning of subnetworks instead of full models, and by targeting common subnetworks which remain identifiable across trianing configurations. Future work involves a deeper theoretical analysis into this paper's findings and performing similar experiments for multimodal and diffusion models.

2 Experiments

Originally, I wanted to perform similar experiments for publicly available multimodal and diffusion models but I vastly underestimated how much compute doing RL requires. Instead, I opted to do an ablation study to see how sensitive the model outputs are to changes in the subnetwork. This relates to the paper because the paper mentioned how the performance of the subnetwork-finetuned model on the test set can exceed that of the full model but didn't explore how sensitive the model is to changes in the subnetwork.

Experimental setup. Following the conventional three-stage pipeline, I found publicly available checkpoints of the Qwen2.5 Math model in the SFT and RL stages of training. Although not from the same source, I assumed for convenience that the Instruct model was the result of subsequent RL finetuning on the SFT model. I tested different tolerance values to see which produced an interesting analysis and I settled for the value $1e{-}2$. I defined the subnetwork to be the set of parameters such that $|\theta_{\text{RL}} - \theta_{\text{SFT}}| >$ tolerance. I created new models by masking particular weights to those of the base Qwen model. The models produced were made by (1) resetting the weights of those corresponding to the subnetwork, (2) resetting the weights of those outside of the subnetwork, and (3) randomly setting the same number of masked weights per layer back to base weights. These three models and the base Qwen2.5 Math and Instruct models were given the following three prompts and the outputs were qualitatively analyzed by hand. The code can be found on github.

Prompts

The three prompts to the math models are given below.

1. "$4x + 5 = 6x + 7.\ x =$",
2. "$x + 2y = 1, 3x + 2y + 4z = 7, -2x + y - 2z = -1.\ (x, y, z) =$",
3. "$\int (w^{\frac{1}{3}} + 10w^{\frac{3}{5}})dw =$"

Results. These experiments were meant to corroborate the findings of the paper but I had mixed results (do note the small sample size though). For the first prompt, all models produced the correct output but the altered models repeated markdown code blocks in its output while the RL finetuned model didn't. The second prompt produced similar results to that of the first prompt except that the RL finetuned model couldn't complete its answer and got stuck in the same erroneous thought loop. For the final prompt, all three model outputs were correct and essentially produced the same output.

Discussion. The altered models had extraneous markdown code blocks in its output which would make sense as it hasn't been finetuned to produce more human-readable outputs. All three altered models produced fairly similar outputs relative to one another, even for the models produced from resetting the subnetwork weights, which is somewhat surprising as the paper hinted towards the importance of the subnetwork. This could just be due to the fact that the Instruct model isn't exactly the subsequent RL finetuned model of the SFT model but this assumption was necessary due to the computational cost of manually performing SFT and RL on models of this scale and the limited publicly available checkpoints for smaller, manageable models. In addition, this is a small sample size so it could just be variance in outputs across runs. A more controlled environment would, obviously, be better for analysis.

References

[1] Sagnik Mukherjee, Lifan Yuan, Dilek Hakkani-Tur, and Hao Peng. Reinforcement learning finetunes small subnetworks in large language models. *arXiv preprint arXiv:2505.11711*, 2025.